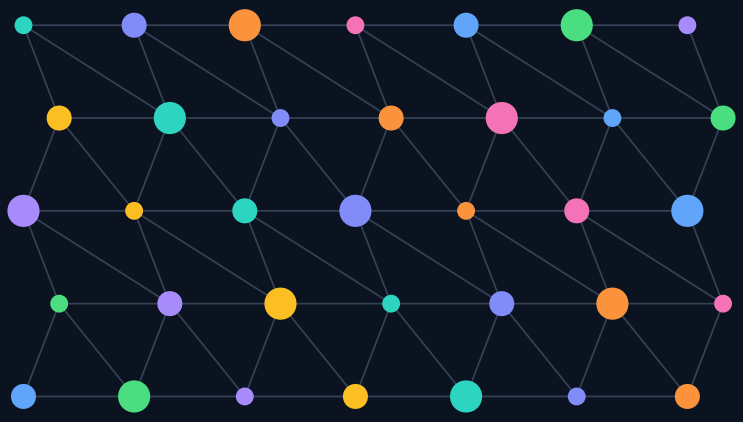
GENERATIVE AI SERIES

LLM Fine-Tuning

A practical field guide to SFT, LoRA/QLoRA and preference alignment: what to use, when, and how to avoid the common traps.



1 Why and When to Fine-Tune
2 The Fine-Tuning Landscape
3 PEFT: LoRA, QLoRA and Friends
4 Alignment: RLHF, DPO and Beyond
5 Data: The Real Lever
6 The Training Recipe
7 Evaluation and Failure Modes
8 Deployment and Checklist
9 Interview Q&A: Concepts
10 Interview Q&A: LLMOps and Scenarios
11 Quick Reference

SFT, LoRA/QLoRA, DPO and friends, evaluation, LLMOps and interview prep

01 Why and When to Fine-Tune

Fine-tuning changes behavior. Retrieval supplies knowledge. Know which problem you have.

A pretrained LLM is a generalist. **Fine-tuning** continues training on your own examples so the weights themselves shift toward a target behavior: a tone, a strict output format, a specialized skill, or reliable tool calling. It is powerful, but it is also the most expensive lever you can pull, so it should come after cheaper options have been tried.

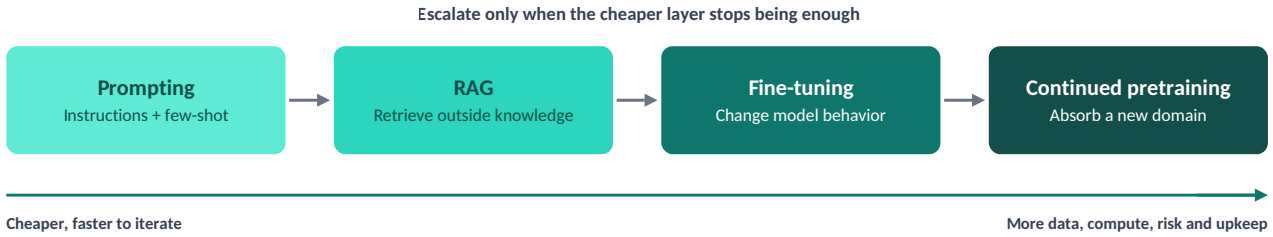


Figure 1. The adaptation ladder: climb only as far as the problem demands.

Approach	Reach for it when	Weak spot
Prompting	Behavior can be described in instructions or a handful of examples; you need fast iteration.	Long prompts cost latency and tokens; brittle on strict formats.
RAG	Answers must use private, fresh or citable knowledge that lives outside the weights.	Does not teach style or skills; quality depends on retrieval.
Fine-tuning	You need consistent tone, structured output, domain-specific skill, dependable tool use, or a small model that imitates a large one.	Poor at injecting facts; needs data, evals and retraining on change.
Continued pretraining	You have a large unlabeled corpus in a new domain or language with unfamiliar vocabulary.	Highest compute cost; still needs SFT afterward.

> **Rule of thumb.** RAG gives the model *knowledge*; fine-tuning changes its *behavior*. They are complements, and many production systems use both: a fine-tuned model that is good at using retrieved context.

Good reasons to fine-tune

- **Format reliability:** always emit valid JSON, a fixed schema, or a house style without a long prompt.
- **Latency and cost:** bake instructions into weights so prompts shrink, or distill a large model into a small one.
- **Specialized skill:** classification, extraction, domain reasoning, or code in an in-house DSL.
- **Tool use and agents:** teach the model when and how to call your functions.

Bad reasons to fine-tune

- Expecting the model to memorize a knowledge base that changes weekly. Use retrieval.
- Having no evaluation set. Without one you cannot tell improvement from regression.
- Fixing a problem that a better prompt or a stronger base model would solve in an hour.

02 The Fine-Tuning Landscape

From raw text to an aligned, deployed model: the stages and the choices inside each.

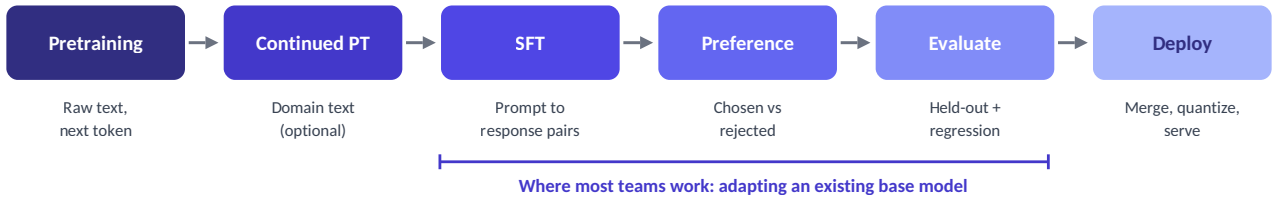


Figure 2. The model lifecycle. Fine-tuning usually means the middle stages.

Stage	Objective	Data	Typical output
Continued pretraining	Next-token prediction on domain text	Large unlabeled corpus	Domain-aware base model
SFT (instruction tuning)	Imitate target responses given prompts	Hundreds to tens of thousands of curated pairs	Instruction-following model
Preference optimization	Prefer better answers over worse ones	Chosen/rejected pairs or ratings	Better aligned, safer, preferred style
RL with verifiable rewards	Maximize a programmatic reward (tests pass, answer correct)	Prompts plus a checker	Stronger reasoning on math and code

Full fine-tuning vs parameter-efficient methods

Full fine-tuning updates every weight. It offers the most capacity but needs memory for weights, gradients and optimizer states (roughly 16 bytes per parameter with mixed-precision AdamW), and produces a complete new model copy for every task. **Parameter-efficient fine-tuning (PEFT)** freezes the base model and trains a tiny set of added parameters, usually under 1% of the total. For most teams PEFT recovers the large majority of the quality at a fraction of the cost.

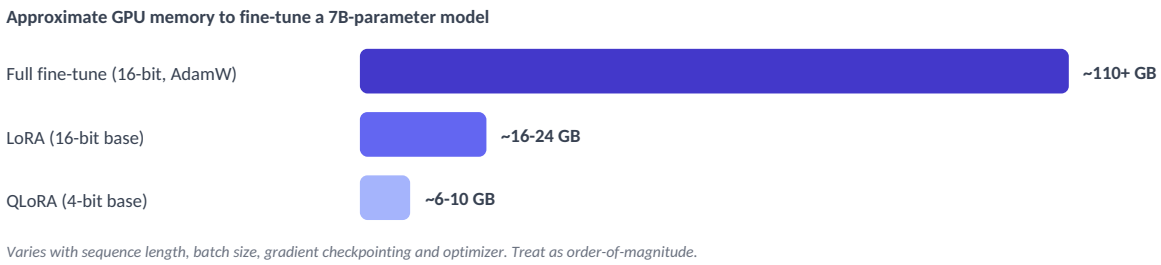


Figure 3. Why PEFT democratized fine-tuning: a 7B model fits on a single consumer or mid-range GPU with QLoRA.

	Full fine-tuning	PEFT (LoRA / QLoRA)
Trainable params	100%	Typically 0.1% to 2%
Memory	Very high	Low to moderate
Artifact per task	Full model copy (many GB)	Adapter file (tens to hundreds of MB)
Forgetting risk	Higher	Lower, since base weights stay frozen
Best when	Large data, big domain shift, ample compute	Most tasks, limited hardware, many variants

03 PEFT: LoRA, QLoRA and Friends

Train a sliver of the model and keep nearly all of the quality.

LoRA: low-rank adaptation

The insight behind LoRA is that the *change* a fine-tune makes to a weight matrix has low intrinsic rank. Instead of learning a full update, LoRA freezes the pretrained matrix `W` and learns two thin matrices whose product approximates the update. `B` starts at zero, so training begins exactly at the base model's behavior.

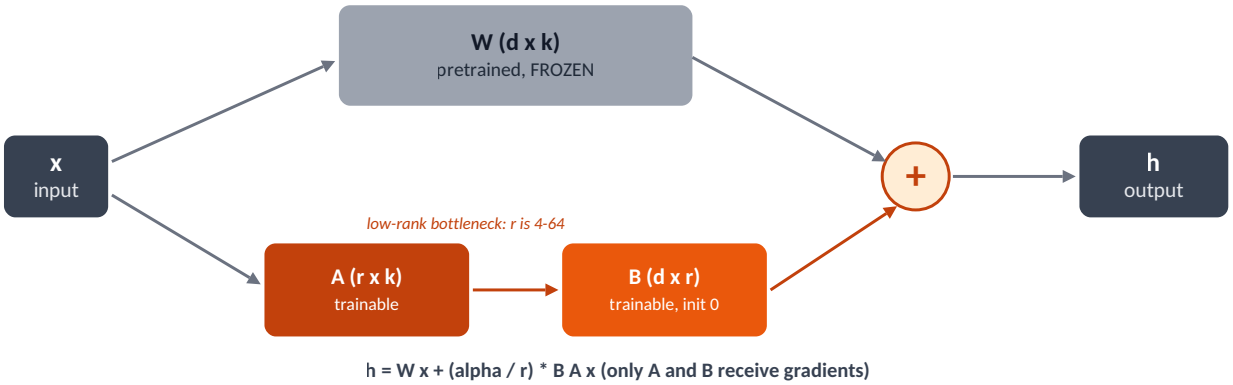


Figure 4. LoRA adds a trainable low-rank path alongside each frozen weight matrix.

```
W' = W + (alpha / r) * B * A with B in R^(d x r), A in R^(r x k), r << min(d, k)
```

Worked example. For a 4096 x 4096 projection, the full matrix has 16,777,216 values. At rank `r = 16`, LoRA trains 16 x (4096 + 4096) = 131,072, which is about 0.78% of that matrix.

Knob	What it does	Starting point
r (rank)	Capacity of the update. Higher fits more complex changes but costs memory and can overfit.	8 to 32; go to 64+ only for hard tasks
lora_alpha	Scales the update by alpha / r. Controls effective step size of the adapter.	r or 2 x r
lora_dropout	Regularization on the adapter path.	0.0 to 0.1
target_modules	Which layers get adapters. Attention-only is cheap; all linear layers usually trains better.	All linear layers (q,k,v,o + MLP)

QLoRA: LoRA on a 4-bit base

QLoRA quantizes the frozen base model to 4 bits and trains LoRA adapters on top in higher precision, with gradients flowing through the quantized weights. Three ideas make it work:

- **NF4:** a 4-bit data type designed for normally distributed weights.
- **Double quantization:** quantizes the quantization constants too, saving extra memory.
- **Paged optimizers:** spill optimizer state to CPU memory during spikes to avoid out-of-memory crashes.

Other PEFT methods

Method	Idea	Note
Adapters	Small bottleneck layers inserted inside each transformer block	Adds inference latency, since layers remain at runtime
Prefix tuning	Learned vectors prepended to keys and values at every layer	Reduces usable context length
Prompt tuning	Learned soft tokens at the input embedding only	Works best on larger models
(IA)3	Learned vectors that rescale keys, values and MLP activations	Extremely few parameters
DoRA	Splits weights into magnitude and direction; applies LoRA to direction	Often a small quality gain over LoRA at the same rank
rsLoRA	Scales by alpha / sqrt(r) instead of alpha / r	More stable when using high ranks

Which method should I use?

Your situation	Reasonable choice
Single GPU with 24 GB or less	QLoRA on all linear layers
Quality-critical task, enough VRAM	LoRA or DoRA, 16-bit base, r = 16 to 64
Many task or customer variants on one base model	One LoRA adapter per variant, served with multi-LoRA
Large dataset and a big domain shift	Full fine-tuning, or continued pretraining then LoRA
Quick, tiny-budget experiments	(IA)3 or prompt tuning, then graduate to LoRA if promising

> **Practical default.** Start with QLoRA or LoRA on all linear layers, r = 16, alpha = 32. Only move to higher rank or full fine-tuning if a well-run experiment shows the adapter is the bottleneck.

04 Alignment: RLHF, DPO and Beyond

SFT teaches the model what to say. Preference tuning teaches it what is better.

After SFT, a model can follow instructions but may still be verbose, unsafe, or inconsistent. **Preference optimization** trains on comparisons: for the same prompt, a *chosen* answer and a *rejected* one. The model learns to move probability toward what humans or a judge prefer.

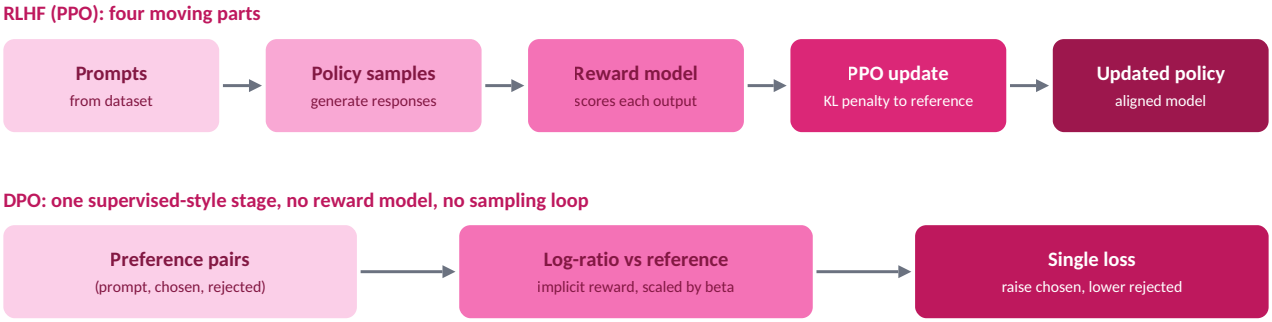


Figure 5. Classic RLHF needs a reward model and a sampling loop; DPO folds the preference signal into one loss.

Choosing a method

Method	How it works	Needs	Pick it when
RLHF (PPO)	Train a reward model on preferences, then optimize the policy against it with a KL penalty to a reference model	Reward model, reference, sampling infra	You need maximum control and have the engineering budget
DPO	Optimizes preference pairs directly; implicit reward is the log-ratio vs a frozen reference	Preference pairs, reference model	Default choice: simple, stable, widely supported
ORPO	Adds an odds-ratio preference penalty to the SFT loss, so SFT and alignment happen in one stage	Preference pairs, no reference	You want to save a stage and memory
KTO	Learns from single thumbs-up or thumbs-down labels instead of pairs	Binary feedback	Your data is unpaired user feedback
SimPO	Reference-free, length-normalized implicit reward	Preference pairs	You want DPO-like results with less memory
GRPO	Samples a group of answers per prompt and scores each relative to the group; no value model	Prompts plus a reward function or checker	Reasoning tasks with verifiable answers (math, code)

DPO in one line

```
loss = - log sigmoid( beta * [ log(pi(y_w|x)/ref(y_w|x)) - log(pi(y_l|x)/ref(y_l|x)) ] )
```

Here `y_w` is the chosen and `y_l` the rejected response. The hyperparameter **beta** (commonly 0.1) controls how far the policy may drift from the reference: lower values allow bigger shifts, higher values keep it closer.

> **Watch for.** Preference methods can reward **length** and **flattery** rather than quality. Balance response lengths in your pairs, and always run SFT first so the policy starts near the data distribution.

05 Data: The Real Lever

A small, clean, representative dataset beats a large noisy one.

Fine-tuning mostly teaches the model to reproduce the *patterns in your data*, including your mistakes. Research such as LIMA showed that about a thousand carefully curated examples can yield a strong instruction follower, which is why quality dominates quantity.

Record format

Store chat data as role-tagged messages and let the tokenizer apply the model's own chat template, so training and inference formatting match exactly.

```
{"messages": [
  {"role": "system",    "content": "You are a concise support assistant."},
  {"role": "user",      "content": "Reset steps for a locked account?"},
  {"role": "assistant", "content": "1. Open Settings > Security. 2. Choose Reset..."}
]}
```

How much data

Goal	Rough scale	Comment
Tone, style, output format	A few hundred to 2,000 examples	Diversity of prompts matters more than count
Domain skill (extraction, classification)	1,000 to 20,000	Cover edge cases and negative examples
General instruction following	10,000 to 100,000+	Mix sources to preserve breadth
Preference alignment	5,000 to 100,000 pairs	Pairs must differ in quality, not just length

Data quality checklist

- **Deduplicate** exact and near-duplicate examples; repeats cause memorization.
- **Mask the loss** to assistant tokens only, so the model does not learn to imitate user prompts.
- **Match production:** prompts should look like real traffic, including messy and adversarial inputs.
- **Verify labels:** spot-check a random sample by hand; automate checks for format validity.
- **Hold out first:** split train, validation and test *before* any augmentation, and decontaminate test items from training data.
- **Blend in general data** (a few percent to 20%) if you must preserve broad abilities.

Synthetic data

A strong model can generate or rewrite training examples (distillation, self-instruct, rejection sampling). It scales well but inherits the teacher's errors and biases, so filter with rules, a judge model and human review. Check the teacher model's license and terms before using its outputs to train another model.

06 The Training Recipe

A reproducible QLoRA SFT run with the Hugging Face stack, plus sane hyperparameters.

Step by step

- **1. Define success:** pick metrics and build the eval set before training anything.
- **2. Baseline:** measure the base model with your best prompt. This is the number to beat.
- **3. Prepare data:** clean, deduplicate, apply the chat template, split.
- **4. Train small first:** a short run on a data subset to catch format and masking bugs.
- **5. Scale and tune:** adjust learning rate, rank and epochs one at a time; watch validation loss.
- **6. Evaluate and iterate:** compare against the baseline, inspect failures, fix the data.

```
import torch
from datasets import load_dataset
from peft import LoraConfig
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from trl import SFTConfig, SFTTrainer

model_id = "your-base-model"                    # any causal LM with a chat template
bnb = BitsAndBytesConfig(                       # QLoRA: 4-bit NF4 base weights
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.bfloat16)
model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb)

lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules="all-linear", task_type="CAUSAL_LM")

args = SFTConfig(output_dir="out", num_train_epochs=2, learning_rate=2e-4,
                 per_device_train_batch_size=4, gradient_accumulation_steps=4,
                 lr_scheduler_type="cosine", warmup_ratio=0.03, bf16=True,
                 logging_steps=10, eval_strategy="steps", eval_steps=100)

ds = load_dataset("json", data_files={"train": "train.jsonl", "test": "val.jsonl"})
trainer = SFTTrainer(model=model, args=args, peft_config=lora,
                     train_dataset=ds["train"], eval_dataset=ds["test"])
trainer.train()
trainer.save_model("out/adapter")               # saves only the small LoRA adapter
```

Hyperparameter starting points

Parameter	LoRA / QLoRA SFT	Full SFT	DPO
Learning rate	1e-4 to 2e-4	1e-5 to 2e-5	~5e-6 (LoRA), ~5e-7 (full)
Epochs	1 to 3	1 to 3	1 to 2
Effective batch size	16 to 64	32 to 128	32 to 64
Scheduler / warmup	Cosine, 3% to 10%	Cosine, 3% to 10%	Cosine or linear, ~10%
Optimizer / precision	AdamW (paged 8-bit for QLoRA), bf16	AdamW, bf16	AdamW, bf16
Method-specific	r = 16, alpha = 32	Gradient checkpointing	beta = 0.1

> **These are starting points, not rules.** Datasets differ. If training loss falls while validation loss rises, you are overfitting: cut epochs, lower the learning rate, or add data. If both stay flat, check masking, templates and learning rate.

07 Evaluation and Failure Modes

If you cannot measure it, you cannot fine-tune it.

Layers of evaluation

Layer	What it tells you	Tip
Task test set	Did the model improve at the thing you trained for?	Frozen, held-out, representative of real traffic
Automatic metrics	Exact match, F1, schema validity, unit tests for code	Prefer programmatic checks wherever possible
LLM-as-judge	Scalable quality scoring on open-ended output	Use a rubric; randomize order; beware position and length bias
Regression suite	Did general ability, safety and refusals degrade?	Run a small general benchmark before and after
Human review	Subtle quality, tone and trust issues	Sample failures and wins; calibrate your judge against humans

Common failure modes

Problem	Symptom	Fix
Catastrophic forgetting	Great on the task, worse at general skills	Use PEFT, lower learning rate, blend in general data
Overfitting	Train loss drops, validation rises; memorized phrases	Fewer epochs, more diverse data, higher dropout
Template mismatch	Odd outputs, runaway generation, missing stop token	Use the model's own chat template in train and inference
Hallucination increase	Confident wrong facts after training on new facts	Fine-tune for behavior; keep facts in retrieval
Repetition or format collapse	Loops, truncated or malformed outputs	Check EOS handling, sequence length, label masking
Preference over-optimization	Verbose, sycophantic, or gamed outputs	Raise beta, balance lengths, audit pairs
Data leakage	Suspiciously high test scores	Deduplicate and decontaminate against the test set

> **Compare fairly.** Evaluate the fine-tuned model against the **best-prompted base model**, not a naive prompt. If the gap is small, the cheaper approach wins.

08 Deployment and Checklist

Ship the adapter, watch it in production, and keep a rollback ready.

From adapter to endpoint

Step	Options	Notes
Keep or merge	Serve base + adapter, or merge with `merge_and_unload()`	Merging removes adapter overhead; keeping adapters allows swapping
Quantize	GPTQ, AWQ, GGUF, FP8	Re-run your eval after quantizing; some tasks are sensitive
Serve	vLLM, TGI, llama.cpp, managed endpoints	Multi-LoRA serving can host many adapters on one base model
Version	Tag data, code, base model and adapter together	Reproducibility makes rollbacks safe
Monitor	Quality samples, latency, cost, drift, safety flags	Feed hard production cases back into the next dataset

Fine-tuning checklist

Before training

- [ ] Tried prompting and RAG first
- [ ] Success metric defined
- [ ] Held-out test set frozen
- [ ] Baseline measured
- [ ] Data deduplicated and spot-checked
- [ ] Chat template and loss masking verified

During training

- [ ] Smoke test on a small subset
- [ ] Validation loss tracked
- [ ] Checkpoints saved regularly
- [ ] One hyperparameter changed at a time
- [ ] Seeds and configs logged

Before release

- [ ] Beats best-prompted baseline
- [ ] General regression suite passed
- [ ] Safety and refusal behavior checked
- [ ] Quantized model re-evaluated
- [ ] Rollback plan ready

After release

- [ ] Sampling production outputs
- [ ] Latency and cost tracked
- [ ] Failure cases collected
- [ ] Retraining cadence decided
- [ ] Adapter and data versions recorded

> **The one-paragraph version.** Start with prompting and RAG. If you still need new behavior, build a small clean dataset and an eval set, run QLoRA SFT with default settings, compare against a well-prompted base model, add DPO only if preference quality matters, and ship the adapter with monitoring and a rollback path.

09 Interview Q&A: Concepts

Short, accurate answers you can say out loud in under a minute each.

1. When would you fine-tune instead of using RAG or better prompts?

Fine-tune for **behavior**: consistent format or tone, a specialized skill, reliable tool calling, or lower latency and cost through shorter prompts and distillation. Use RAG for fresh, private or citable **knowledge**. I try prompting, then RAG, and compare any fine-tune against the best-prompted baseline. Often the answer is both.

2. Explain LoRA and why it works.

Freeze the pretrained weight `W` and learn an update `(alpha/r)*B*A` with rank `r`. Task adaptation tends to need a low-rank change, so a small factorization captures most of it. `B` starts at zero, so training begins at base behavior. It trains under 1% of parameters, produces small adapter files, and can be merged into `W` for zero inference overhead.

3. What do r and alpha do, and how do you set them?

`r` sets adapter capacity; `alpha` scales the update by `alpha/r`. I start with r = 16, alpha = 32 on all linear layers, raise r if the model underfits, and tune learning rate before touching alpha.

4. What does QLoRA quantize, and what stays in higher precision?

The frozen base weights are stored in 4-bit NF4 (with double quantization) and dequantized on the fly for compute. The LoRA adapters and their gradients stay in bf16 or fp16. Paged optimizers absorb memory spikes. A 7B model then trains in roughly 6 to 10 GB.

5. SFT vs RLHF vs DPO?

SFT imitates demonstrations. RLHF trains a reward model on preferences, then optimizes the policy with PPO under a KL penalty. DPO optimizes chosen/rejected pairs directly against a frozen reference, with no reward model or sampling loop. The common recipe is SFT, then DPO.

6. What does beta do in DPO?

It controls how far the policy may move from the reference. Lower beta allows larger shifts; higher beta keeps the model conservative. 0.1 is a typical start.

7. How do you prevent catastrophic forgetting?

Use PEFT, a low learning rate and few epochs; mix in general data (replay); and run a regression suite before and after training. Stop early when validation quality peaks.

8. Why mask the loss to the assistant tokens, and why use the chat template?

Masking makes the model learn to produce answers rather than imitate prompts. Using the model's own chat template, including the end-of-turn token, keeps training and inference formatting identical; mismatches cause odd outputs and runaway generation.

9. Training loss falls but quality is poor. What do you check?

Overfitting or memorization (validation loss rising), label noise, template or masking bugs, and a mismatch between loss and the real task metric. Loss is a proxy, so I rely on a task-specific eval set.

10. Why is the LoRA learning rate higher than for full fine-tuning?

Adapters are few and `B` starts at zero, so they need larger steps to move the output meaningfully (about 1e-4 to 2e-4). Full fine-tuning changes every weight, so it needs small steps (about 1e-5 to 2e-5) to avoid instability and forgetting.

10 Interview Q&A: LLMOps and Scenarios

What an LLM Operations Engineer is expected to own after the model is trained.

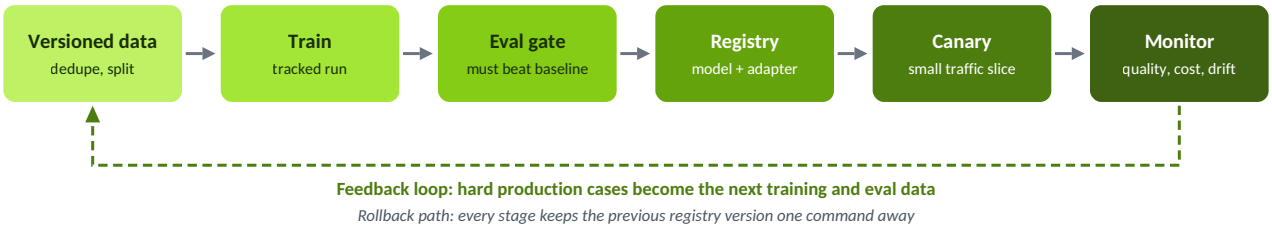


Figure 6. A safe retrain-and-ship loop with an evaluation gate and rollback.

11. Design a pipeline to retrain and ship fine-tuned models safely.

Version data and configs; train as a tracked run; pass an automated eval gate (task metrics plus regression and safety suites) against the current production model; register model and adapter together; release via shadow or canary traffic; monitor; and feed hard cases back into data. Rollback is a registry pointer change.

12. How do you serve many fine-tuned variants cheaply?

Keep one base model in GPU memory and hot-swap LoRA adapters per request (for example vLLM multi-LoRA or LoRAX), batching across adapters. Merging each variant would cost a full model copy of memory per variant.

13. What do you monitor in production?

Sampled quality (LLM-judge with human calibration), task success rate, latency p50/p95, tokens per second, cost per request, GPU utilization, input drift, refusal and safety rates, and user feedback.

14. When do you retrain?

On evidence, not a calendar: a quality drop against the baseline, input drift, new product requirements, a new base model release, or enough new high-quality feedback. Every candidate must pass the same eval gate.

15. How do you reduce inference cost of a fine-tuned model?

Quantize (AWQ, GPTQ, FP8), merge adapters, distill into a smaller model, bake instructions into weights to shorten prompts, use continuous batching, prefix and KV caching, speculative decoding, and route easy requests to a smaller model.

16. Scenario: it scores well offline but badly in production. Why?

Likely causes: a different input distribution, a chat template or system prompt mismatch, quantization loss, sampling settings that differ, a tokenizer version change, or test-set leakage that inflated the offline score. I diff the exact production request against the eval format first.

17. Scenario: safety and refusal behavior got worse after fine-tuning. What now?

Fine-tuning can erode safety alignment even on benign data. Add refusal and safety examples to the mix, run a safety suite as a release gate, prefer PEFT with a low learning rate, consider preference tuning on safety pairs, and keep a guardrail layer in front of the model.

18. How do you make experiments reproducible?

Log dataset version or hash, base model revision, full config, seed, library versions, metrics and artifacts in a tracker such as MLflow or Weights and Biases, and promote through a model registry so any production model maps back to its exact inputs.

11 Quick Reference

Numbers, terms and a project-story template to review right before the interview.

Numbers worth remembering

Item	Value
Full fine-tune memory	About 16 bytes per parameter with mixed-precision AdamW, so roughly 112 GB for 7B
Weights only	fp16 is 2 bytes per parameter (7B is about 14 GB); 4-bit is about 0.5 bytes (about 3.5 GB)
LoRA parameters per matrix	r x (d + k); at r = 16 on 4096 x 4096 that is 131,072 (about 0.78%)
Effective batch size	Per-device batch x gradient accumulation steps x number of GPUs
Typical learning rates	LoRA 1e-4 to 2e-4; full SFT 1e-5 to 2e-5; DPO about 5e-6 (LoRA) or 5e-7 (full)
Defaults to quote	r = 16, alpha = 32, 1 to 3 epochs, cosine schedule, DPO beta = 0.1

Glossary

Term	Meaning	Term	Meaning
SFT	Supervised fine-tuning on prompt/ response pairs	PEFT	Train a small set of added parameters, freeze the base
LoRA	Low-rank update B*A added to frozen weights	QLoRA	LoRA on a 4-bit quantized base
NF4	4-bit data type suited to normal weights	DoRA	LoRA on direction, separate magnitude
RLHF	Reward model plus RL (PPO) from human preferences	DPO	Direct preference loss, no reward model
KL penalty	Keeps policy close to the reference model	Reference model	Frozen copy used as the anchor in DPO and RLHF
Chat template	Model-specific format for roles and special tokens	Loss masking	Compute loss only on assistant tokens
Grad accumulation	Sum gradients over steps to mimic a larger batch	Grad checkpointing	Recompute activations to save memory
Forgetting	Loss of general ability after narrow training	Distillation	Train a small model on a large model's outputs
Merge adapter	Fold LoRA weights into the base for serving	Perplexity	exp(mean negative log-likelihood) of text

Tell me about a fine-tuning project: a 60-second structure

- **Problem:** what behavior was missing and why prompting or RAG was not enough.
- **Baseline:** the best-prompted model and its score.
- **Data:** source, size, cleaning, split, and how leakage was avoided.
- **Method:** base model, LoRA or QLoRA settings, and why that choice.
- **Evaluation:** task metrics, regression and safety checks, and the final gain.
- **Operations:** deployment, monitoring, cost, and what you would improve next.
- **Tip:** lead with the decision logic (why fine-tune, how you measured), not tool names.